

ЛАБОРАТОРНА РОБОТА № 3
REST API з Express.js та інтеграційне тестування

Мета: Метою цієї лабораторної роботи є побудова RESTful API на Express.js з TypeScript, валідація вхідних даних за допомогою Zod, використання зберігання в пам'яті на основі Map, розподіл app.ts та server.ts, а також написання інтеграційних тестів із Supertest.

Хід роботи:

Завдання 1. Ініціалізація проєкту

Створено проєкт Lab03 з пакетами express, cors, zod, typescript, jest, ts-jest, supertest та npm-скриптами: dev, build, test, test:coverage.

Структура проєкту:

```
Lab03/  
  src/schemas/entity.schema.ts  
  src/storage/entity.ts  
  src/routes/entity.ts  
  src/middleware/validate.ts, errorHandler.ts  
  src/app.ts, server.ts  
  tests/entity.test.ts
```

Завдання 2. Схеми сутностей та типи (Movie)

Сутність: Movie. Поля: title (обов'язкове), description (необов'язкове), releaseYear, genre (enum), director. updateSchema робить усі поля необов'язковими. Тип Entity виводиться з createSchema та розширюється полями id, createdAt, updatedAt.

Лістинг src/schemas/entity.schema.ts:

```
import { z } from 'zod';  
export const genres = [  
  'drama',  
  'comedy',  
  'action',  
  'sci-fi',  
  'horror',  
  'documentary',  
] as const;  
const currentYear = new Date().getFullYear();  
  
export const createSchema = z.object({  
  title: z.string().min(1).max(100),  
  description: z.string().max(500).optional(),  
  releaseYear: z.number().min(1880).max(currentYear + 2),  
  genre: z.enum(genres),  
  director: z.string().min(1).max(100).optional(),  
});
```

					ДУ «Житомирська політехніка».26.121.20.000 - Лр3									
					DU «Zhytomyrska politehnika».26.121.20.000 - Lr3									
Змн.	Арк.	Release year: Z.number (mm), mm (188)	№ докум.	Підпис	Дата									
Розроб.		Список: Z.number (mm), mm (188)	Риженко Я.В.			Звіт з лабораторної роботи				Літ.	Арк.	Аркушів		
Перевір.			Фуріхата Д.В.									1	11	
Керівник					ФІКТ Гр. ІПЗ-23-1									
Н. контр.														
Зав. каф.														

```

    director: z.string().min(1).max(100),
  });

export const updateSchema = createSchema.partial();

export const filterSchema = z.object({
  genre: z.enum(genres).optional(),
  minYear: z.coerce.number().int().min(1888).optional(),
  maxYear: z.coerce.number().int().max(currentYear + 2).optional(),
  title: z.string().min(1).optional(),
});

export type CreateInput = z.infer<typeof createSchema>;
export type UpdateInput = z.infer<typeof updateSchema>;
export type FilterInput = z.infer<typeof filterSchema>;

export type Entity = CreateInput & {
  id: string;
  createdAt: Date;
  updatedAt: Date;
};

```

Завдання 3. Зберігання в оперативній пам'яті

storage/entity.ts використовує Map<string, Entity> з методами getAll, getById, create, update, remove, reset, getRecent та фільтрацією на рівні сховища.

Лістинг src/storage/entity.ts:

```

import type { CreateInput, Entity, FilterInput, UpdateInput } from '../schemas/entity.schema';

const store = new Map<string, Entity>();

export function getAll(filters?: FilterInput): Entity[] {
  let items = Array.from(store.values());

  if (filters?.genre) {
    items = items.filter((movie) => movie.genre === filters.genre);
  }
  if (filters?.minYear !== undefined) {
    items = items.filter((movie) => movie.releaseYear >= filters.minYear!);
  }
  if (filters?.maxYear !== undefined) {
    items = items.filter((movie) => movie.releaseYear <= filters.maxYear!);
  }
  if (filters?.title) {
    const query = filters.title.toLowerCase();
    items = items.filter((movie) => movie.title.toLowerCase().includes(query));
  }

  return items;
}

export function getRecent(): Entity[] {
  const cutoffYear = new Date().getFullYear() - 5;
  return Array.from(store.values()).filter((movie) => movie.releaseYear >= cutoffYear);
}

```

		Рижченко Я.В.			ДУ «Житомирська політехніка». 26.121.20.000 – ЛрЗ	Арк.
		Фуріхата Д.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		2

```

}

export function getById(id: string): Entity | undefined {
  return store.get(id);
}

export function create(data: CreateInput): Entity {
  const now = new Date();
  const entity: Entity = {
    ...data,
    id: crypto.randomUUID(),
    createdAt: now,
    updatedAt: now,
  };
  store.set(entity.id, entity);
  return entity;
}

export function update(id: string, data: UpdateInput): Entity | undefined {
  const existing = store.get(id);
  if (!existing) {
    return undefined;
  }

  const updated: Entity = {
    ...existing,
    ...data,
    updatedAt: new Date(),
  };
  store.set(id, updated);
  return updated;
}

export function remove(id: string): boolean {
  return store.delete(id);
}

export function reset(): void {
  store.clear();
}

```

Завдання 4. CRUD-маршрути та проміжне ПЗ

validate(schema) розбирає req.body за допомогою Zod. errorHandler повертає 400 для ZodError (err.issues) та 500 для інших помилок. Маршрути: GET/POST /api/movies, GET/PATCH/DELETE /api/movies/:id, GET /api/movies/recent.

Лістинг src/middleware/validate.ts:

```

import { NextFunction, Request, Response } from 'express';
import { ZodSchema } from 'zod';

export function validate(schema: ZodSchema) {
  return (req: Request, _res: Response, next: NextFunction): void => {
    try {

```

		Рижченко Я.В.			ДУ «Житомирська політехніка».26.121.20.000 – ЛрЗ	Арк.
		Фуріхата Д.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		3

```

    req.body = schema.parse(req.body);
    next();
  } catch (err) {
    next(err);
  }
};
}

```

Лістинг src/middleware/errorHandler.ts:

```

import { NextFunction, Request, Response } from 'express';
import { ZodError } from 'zod';

```

```

export function errorHandler(
  err: Error,
  _req: Request,
  res: Response,
  _next: NextFunction,
): void {
  if (err instanceof ZodError) {
    res.status(400).json({
      error: 'Validation failed',
      details: err.issues,
    });
    return;
  }

```

```

  console.error(err);
  res.status(500).json({
    error: 'Internal server error',
    message: err.message,
  });
}

```

Лістинг src/routes/entity.ts:

```

import { NextFunction, Request, Response, Router } from 'express';
import { createSchema, filterSchema, updateSchema } from '../schemas/entity.schema';
import { validate } from '../middleware/validate';
import * as storage from '../storage/entity';

```

```

const router = Router();

```

```

router.get('/recent', (_req: Request, res: Response) => {
  res.json(storage.getRecent());
});

```

```

router.get('/', (req: Request, res: Response, next: NextFunction) => {
  try {
    const filters = filterSchema.parse(req.query);
    res.json(storage.getAll(filters));
  } catch (err) {
    next(err);
  }
});

```

```

router.get('/:id', (req: Request<{ id: string }>, res: Response) => {
  const movie = storage.getById(req.params.id);

```

		Рижченко Я.В.			ДУ «Житомирська політехніка».26.121.20.000 – Лр3	Арк.
		Фуріхата Д.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		4

```

    if (!movie) {
      res.status(404).json({ error: 'Movie not found' });
      return;
    }
    res.json(movie);
  });

  router.post('/', validate(createSchema), (req: Request, res: Response) => {
    const movie = storage.create(req.body);
    res.status(201).json(movie);
  });

  router.patch('/:id', validate(updateSchema), (req: Request<{ id: string }>, res: Response) => {
    const movie = storage.update(req.params.id, req.body);
    if (!movie) {
      res.status(404).json({ error: 'Movie not found' });
      return;
    }
    res.json(movie);
  });

  router.delete('/:id', (req: Request<{ id: string }>, res: Response) => {
    const deleted = storage.remove(req.params.id);
    if (!deleted) {
      res.status(404).json({ error: 'Movie not found' });
      return;
    }
    res.status(204).send();
  });

  export default router;

```

Завдання 5. app.ts та server.ts

app.ts налаштовує Express без listen(). server.ts імпортує app та запускає сервер.

Лістинг src/app.ts:

```

import cors from 'cors';
import express from 'express';
import { errorHandler } from './middleware/errorHandler';
import entityRouter from './routes/entity';

const app = express();

app.use(cors());
app.use(express.json());
app.use('/api/movies', entityRouter);
app.use(errorHandler);

export default app;

```

Лістинг src/server.ts:

```

import app from './app';

```

		Рижченко Я.В.			ДУ «Житомирська політехніка».26.121.20.000 – ЛрЗ	Арк.
		Фуріхата Д.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		5

```
const PORT = Number(process.env.PORT) || 3000;

app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});
```

Завдання 6. Інтеграційні тести

tests/entity.test.ts використовує Supertest та beforeEach(storage.reset). 15 тестів охоплюють CRUD, помилки валідації, випадки 404, фільтри запитів та /recent.

Лістинг tests/entity.test.ts:

```
import request from 'supertest';
import app from '../src/app';
import * as storage from '../src/storage/entity';

const sampleMovie = {
  title: 'Inception',
  description: 'A thief who steals corporate secrets through dream-sharing technology.',
  releaseYear: 2010,
  genre: 'sci-fi' as const,
  director: 'Christopher Nolan',
};

const olderMovie = {
  title: 'The Godfather',
  releaseYear: 1972,
  genre: 'drama' as const,
  director: 'Francis Ford Coppola',
};

const recentMovie = {
  title: 'Dune: Part Two',
  releaseYear: new Date().getFullYear(),
  genre: 'sci-fi' as const,
  director: 'Denis Villeneuve',
};

describe('Movies API', () => {
  beforeEach(() => {
    storage.reset();
  });

  describe('POST /api/movies', () => {
    it('creates a movie and returns 201', async () => {
      const res = await request(app).post('/api/movies').send(sampleMovie).expect(201);

      expect(res.body).toMatchObject({
        title: sampleMovie.title,
        genre: sampleMovie.genre,
        director: sampleMovie.director,
        releaseYear: sampleMovie.releaseYear,
      });
      expect(res.body).toHaveProperty('id');
      expect(res.body).toHaveProperty('createdAt');
```

```

    expect(res.body).toHaveProperty('updatedAt');
  });

  it('returns 400 for invalid payload', async () => {
    const res = await request(app)
      .post('/api/movies')
      .send({ title: '', releaseYear: 1800, genre: 'unknown', director: '' })
      .expect(400);

    expect(res.body).toHaveProperty('error', 'Validation failed');
    expect(res.body.details).toBeDefined();
  });
});

describe('GET /api/movies', () => {
  it('returns all movies', async () => {
    await request(app).post('/api/movies').send(sampleMovie);
    await request(app).post('/api/movies').send(olderMovie);

    const res = await request(app).get('/api/movies').expect(200);

    expect(res.body).toHaveLength(2);
  });

  it('filters by genre', async () => {
    await request(app).post('/api/movies').send(sampleMovie);
    await request(app).post('/api/movies').send(olderMovie);

    const res = await request(app).get('/api/movies').query({ genre: 'drama' }).expect(200);

    expect(res.body).toHaveLength(1);
    expect(res.body[0].title).toBe('The Godfather');
  });

  it('filters by minYear', async () => {
    await request(app).post('/api/movies').send(sampleMovie);
    await request(app).post('/api/movies').send(olderMovie);

    const res = await request(app).get('/api/movies').query({ minYear: 2000 }).expect(200);

    expect(res.body).toHaveLength(1);
    expect(res.body[0].title).toBe('Inception');
  });

  it('filters by genre and minYear together', async () => {
    await request(app).post('/api/movies').send(sampleMovie);
    await request(app).post('/api/movies').send(recentMovie);
    await request(app).post('/api/movies').send(olderMovie);

    const res = await request(app)
      .get('/api/movies')
      .query({ genre: 'sci-fi', minYear: 2015 })
      .expect(200);

    expect(res.body).toHaveLength(1);
    expect(res.body[0].title).toBe('Dune: Part Two');
  });
});

```

		Риженко Я.В.			ДУ «Житомирська політехніка». 26.121.20.000 – Пр3	Арк.
		Фуріхата Д.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		7

```

});

it('returns 400 for invalid filter query', async () => {
  const res = await request(app).get('/api/movies').query({ genre: 'invalid' }).expect(400);

  expect(res.body).toHaveProperty('error', 'Validation failed');
});

});

describe('GET /api/movies/recent', () => {
  it('returns movies released within the last 5 years', async () => {
    await request(app).post('/api/movies').send(sampleMovie);
    await request(app).post('/api/movies').send(recentMovie);
    await request(app).post('/api/movies').send(olderMovie);

    const res = await request(app).get('/api/movies/recent').expect(200);

    expect(res.body).toHaveLength(1);
    expect(res.body[0].title).toBe('Dune: Part Two');
  });
});

describe('GET /api/movies/:id', () => {
  it('returns a movie by id', async () => {
    const created = await request(app).post('/api/movies').send(sampleMovie);

    const res = await request(app).get(`/api/movies/${created.body.id}`).expect(200);

    expect(res.body.id).toBe(created.body.id);
    expect(res.body.title).toBe(sampleMovie.title);
  });
});

it('returns 404 when movie does not exist', async () => {
  const res = await request(app)
    .get('/api/movies/00000000-0000-0000-0000-000000000000')
    .expect(404);

  expect(res.body).toHaveProperty('error', 'Movie not found');
});

});

describe('PATCH /api/movies/:id', () => {
  it('updates a movie and returns 200', async () => {
    const created = await request(app).post('/api/movies').send(sampleMovie);

    const res = await request(app)
      .patch(`/api/movies/${created.body.id}`)
      .send({ title: 'Inception (Director\'s Cut)' })
      .expect(200);

    expect(res.body.title).toBe('Inception (Director\'s Cut)');
    expect(new Date(res.body.updatedAt).getTime()).toBeGreaterThanOrEqual(
      new Date(created.body.updatedAt).getTime(),
    );
  });
});

```

		Риженко Я.В.			ДУ «Житомирська політехніка». 26.121.20.000 – ПрЗ	Арк.
		Фуріхата Д.В.				8
Змн.	Арк.	№ докум.	Підпис	Дата		


```

it('returns 404 when updating a missing movie', async () => {
  await request(app)
    .patch('/api/movies/00000000-0000-0000-0000-000000000000')
    .send({ title: 'Ghost' })
    .expect(404);
});

it('returns 400 for invalid update payload', async () => {
  const created = await request(app).post('/api/movies').send(sampleMovie);

  const res = await request(app)
    .patch(`/api/movies/${created.body.id}`)
    .send({ releaseYear: 1700 })
    .expect(400);

  expect(res.body).toHaveProperty('error', 'Validation failed');
});

describe('DELETE /api/movies/:id', () => {
  it('deletes a movie and returns 204', async () => {
    const created = await request(app).post('/api/movies').send(sampleMovie);
    await request(app).delete(`/api/movies/${created.body.id}`).expect(204);

    await request(app).get(`/api/movies/${created.body.id}`).expect(404);
  });

  it('returns 404 when deleting a missing movie', async () => {
    await request(app)
      .delete('/api/movies/00000000-0000-0000-0000-000000000000')
      .expect(404);
  });
});

```

Результат тестування: npm test - 15 пройдено. Покриття коду: близько 93% операторів.

Завдання 7. Фільтри запитів та доменний маршрут

Фільтри: genre, minYear, maxYear, title (комбіновані). Доменний маршрут GET /api/movies/recent повертає фільми, випущені за останні 5 років.

Приклади запитів:

```

npm run dev
curl -X POST http://localhost:3000/api/movies -H "Content-Type: application/json" \
  -d '{"title": "Inception", "releaseYear": 2010, "genre": "sci-fi", "director": "Christopher Nolan"}'
curl http://localhost:3000/api/movies?genre=sci-fi&minYear=2000
curl http://localhost:3000/api/movies/recent

```

Посилання на репозиторій - <https://github.com/DDXwild/node.js-practice>

Висновок: Реалізовано REST API для фільмів із правильними HTTP-статус кодами, валідацією через Zod, зберіганням у пам'яті, фільтрацією на рівні сховища, розподіленим налаштуванням app/server та 15 інтеграційними тестами Supertest. Проєкт готовий до здачі.

		Рижченко Я.В.			ДУ «Житомирська політехніка». 26.121.20.000 – ПрЗ	Арк.
		Фуріхата Д.В.				
Змн.	Арк.	№ докум.	Підпис	Дата		9